

Le patron de conception État

Durée : 1h00

Crédit : « <https://www.math.univ-paris13.fr/~chaussar/> »

1 Instructions générales

1. Téléchargez l'archive zip associée à cet énoncé sur la plateforme pédagogique.
2. Ouvrez le projet dans l'IDE IntelliJ IDEA (ou autre).
3. Si vous ne l'avez pas encore fait, configurez votre IDE pour n'accepter que des attributs avec un identifiant commençant pas un '_'. (*Dans IntelliJ IDEA, menu Settings::Editor::Code Style::Java, ajoutez le caractère '_' dans field et static field. Puis, dans le menu Settings::Editor::inspection::Java::Naming Conventions ajoutez la vérification de Field naming convention (instance field et static field) avec l'expression régulière : `_[a-z][A-Za-z\d]*`.*)
4. Exécutez le programme pour vérifier que tout fonctionne correctement.

2 Données du problème

On souhaite créer un distributeur de billets. Le distributeur peut être vu comme une machine à trois états : en attente d'insertion de carte, en attente d'opération, et en attente de retrait d'espèces. On pourra faire quatre actions sur la machine : insérer une carte, entrer un code, retirer des espèces et retirer la carte. Évidemment, selon l'état de la machine, les actions auront un comportement différent. Par exemple si le distributeur est en attente de carte, l'opération entrer code n'a aucune action.

Voici le squelette d'un code pour le distributeur :

```
public class Distributeur {
    private enum EtatDistributeur {
        ETAT_ATTENTE_CARTE, ETAT_ATTENTE_CODE, ETAT_ATTENTE_OPERATION;
    }

    private EtatDistributeur _etat;
    private Carte _carte;

    public Distributeur() {
        _etat = ETAT_ATTENTE_CARTE;
    }
}
```

```

public void insererCarte( Carte client ) {
    if (_etat == ETAT_ATTENTE_CARTE) {
        System.out.println("Insertion carte Client");
        _carte = client;
    } else if (_etat == ETAT_ATTENTE_CODE) {
        // ...
    } else if (_etat == ETAT_ATTENTE_OPERATION) {
        // ...
    }
}

public void entrerUnCode() {
    if (_etat == ETAT_ATTENTE_CODE) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Entrer le code secret");
        int motDePasse = scanner.nextInt();
        // ...
    } else {
        // ..
    }
}

public void retirerEspeces() { }

public void retirerCarte() { }
}

```

3 Exercice 1

La classe `Carte` est une classe avec un constructeur qui prend en paramètre un code secret.

Écrivez le code de la méthode `estValide()` qui répond si un code est bon.

Si le Client entre trois fois un mauvais code, sa carte doit être avalée. Pour cela, créez en plus une méthode `resteEssais()` qui indique s'il reste des essais possibles.

Codez ensuite mes méthodes du Distributeur.

Programmez un test dans le `main()` pour un cas d'utilisation où le client entre le code après deux essais puis retire 10 euros.

Nous allons donc, avant de rajouter l'état vide, modifier les classes et organiser mieux le projet.

4 Exercice 2

La machine semble bien marcher, mais certains clients se plaignent. En effet, certaines fois, la machine ne donne pas d'argent... Rapidement, les ingénieurs de la machine comprennent que lorsque la machine n'a plus de billets, et elle ne peut pas donner

d'argent.

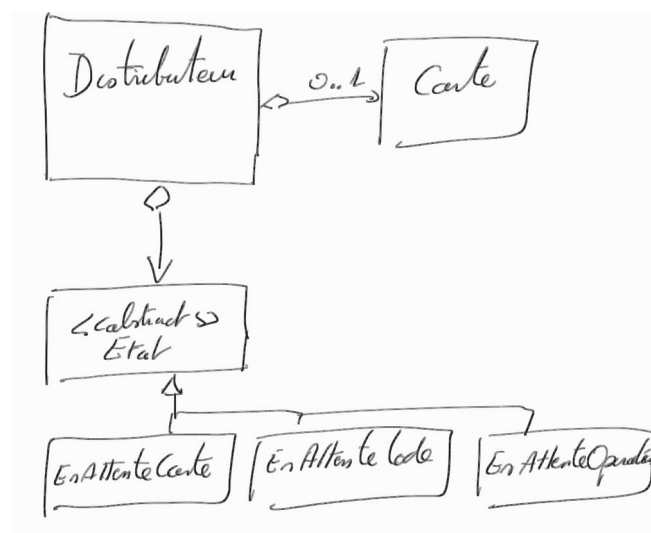
Vous devez donc rajouter un état à la machine, qui sera l'état vide. Cependant, vous voyez rapidement que rajouter un état change beaucoup de code et n'est donc pas très pratique (pas SOLID).

5 Exercice 3

Nous allons refondre la conception pour utiliser le patron de conception État. Créez une classe abstraite `Etat` qui contiendra les quatre méthodes de la machine (insérer la carte, entrer code, retirer argent, reprendre carte). Créez ensuite les trois sous-classes `Etat`, une par état possible de la machine. Redistribuez le code de l'exercice précédent dans les méthodes de ces classes.

Mettez à jour le `Distributeur` pour utiliser des méthodes des sous-classes d'État.

Pour vous aider, voici le diagramme de classe que vous devriez obtenir.



Testez avec un Client que tout fonctionne comme précédemment.

6 Exercice 4

Ajoutez un état `MachineVide` pour empêcher de retirer de l'argent quand la machine est vide. Vous allez donc ajouter au `Distributeur` une variable de classe permettant de connaître son stock d'argent, et vous vérifierez, lorsque vous donnerez de l'argent à un Client, qu'il y a assez d'espèces dans le `Distributeur`. Notez qu'avec cette nouvelle modélisation l'ajout d'un nouvel état est facilité par le patron de conception.

7 Exercice 4

Ajoutez aussi un nouveau service `donnerSolde()`, qui permettra au client d'accéder à

son solde lorsque le Distributeur est en mode en attente d'opération. Notez, là encore, qu'avec cette nouvelle modélisation l'ajout d'un nouveau service est facilité par le patron de conception.